PyGame Tractor Pong Tutorial - Part 7

Contents

PyGame Tractor Pong Tutorial - Part 7 1
Preview of the Game 1
Collision time 2
Draw Tractor 5
Start Music 6
Game Over 7
Update Ball 9
Check Collision 9
Draw the Score 10
What's Next? 12
Assignment Submission 12

Time required: 30 minutes

Preview of the Game

Atari. - the year: 1973 - the date: - November 29th -

That game is called Pong . . . . Then there was Tractor Pong.

Tractor Pong Demo Video

Collision time

The pygame_menu library allows us to create menus.

1. Save **tractor_pong_6.py** as **tractor_pong_7.py**
2. Install pygame-menu-ce

```
# Install pygame-menu-ce
pip install pygame-menu-ce
```

```
# pip install pygame-ce
import pygame
# pip install pygame-menu-ce
import pygame_menu as pm
from sys import exit
from random import randint
from time import sleep
import config
```

```python
class TractorPong:
    # ------------------ INITIALIZE PYGAME ------------------------------#
    def __init__(self):
        # Pre initalize mixer with larger buffer size for better performance
        pygame.mixer.pre_init(
            44100,  # frequency (Hz)
            16,     # bit depth
            2,      # number of channels, 1 mono, 2 stereo
            4096    # buffer size, larger to optimize music playback.
        )

        # Initialize the pygame library
        pygame.init()

        # Create the game surface (window)
        self.surface = pygame.display.set_mode(
            (config.WIDTH, config.HEIGHT)
        )

        # Set window caption
        pygame.display.set_caption("Tractor Pong")

        # CLOCK object manages how fast the game runs
        self.clock = pygame.time.Clock()

        # Only allow these events to be captured
        # This helps optimize the game for slower computers
        pygame.event.set_allowed(
            [
                pygame.QUIT,
                pygame.KEYDOWN
            ]
        )

        self.load_assets()
        self.draw_tractor()
        self.start_music()
```

```python
# ---------------------- LOAD ASSETS -------------------------------------#
    def load_assets(self):
        # Load png image, use as program icon
        self.ball_ico = pygame.image.load(
            "./assets/blue_ball.png").convert_alpha()
        pygame.display.set_icon(self.ball_ico)

        # Load the images from the file system into a variable
        self.ball = pygame.image.load(
            "assets/soccer_ball.png").convert_alpha()
        self.tractor = pygame.image.load(
            "assets/green_tractor.png").convert_alpha()

        # Create a rectangle the same size as the image
        # rect is used to set the location of the image
        self.ball_rect = self.ball.get_rect()
        self.tractor_rect = self.tractor.get_rect()

        # Initial postion of the ball rectangle x random, y/top = 10
        self.set_ball_location()
        self.ball_rect.y = 10

        # Ball speed in pixels for x, y
        self.set_ball_direction()
        self.speed_y = 3
```

Load assets has changed.

```python
# ---------------------- LOAD ASSETS -----------------------------------
    Codiumate: Options | Test this method
    def load_assets(self):
        # Load png image, use as program icon
        self.ball_ico = pygame.image.load(
            "./assets/blue_ball.png").convert_alpha()
        pygame.display.set_icon(self.ball_ico)

        # Load the images from the file system into a variable
        self.ball = pygame.image.load(
            "assets/soccer_ball.png").convert_alpha()
        self.tractor = pygame.image.load(
            "assets/green_tractor.png").convert_alpha()

        # Create a rectangle the same size as the image
        # rect is used to set the location of the image
        self.ball_rect = self.ball.get_rect()
        self.tractor_rect = self.tractor.get_rect()

        # Initial postion of the ball rectangle x random, y/top = 10
        self.set_ball_location()
        self.ball_rect.y = 10

        # Ball speed in pixels for x, y
        self.set_ball_direction()
        self.speed_y = 3

        # Initial location of the tractor
        self.tractor_rect.left = config.WIDTH // 2
        self.tractor_rect.top = config.HEIGHT - 90

        # Speed in pixels for the tractor
        self.tractor_speed = 4

        # Keep track of score
        self.score = 0
```

Draw Tractor

A method has been added to draw the background and tractor before the tractor startup sound starts.

```python
# ---------------------- DRAW TRACTOR ------------------------------------ #
    Codiumate: Options | Test this method
    def draw_tractor(self):
        self.surface.fill(config.COUGAR_GOLD)
        # Draw the tractor on the backbuffer
        self.surface.blit(
            self.tractor,       # Image to draw
            self.tractor_rect   # Location to draw the image
        )
        pygame.display.update()
```

Start Music

This methods loads and starts the background music.

```python
# ---------------------- START MUSIC ------------------------------------- #
    Codiumate: Options | Test this method
    def start_music(self):
        tractor_start = pygame.mixer.Sound("./assets/tractor_starting_up.mp3")
        tractor_start.set_volume(0.4)  # Set volume to 50%
        tractor_start.play()

        # Wait until the sound has finished playing
        while pygame.mixer.get_busy():
            sleep(0.1)  # wait a bit to reduce CPU usage

        self.ball_hit = pygame.mixer.Sound("./assets/whip.wav")
        self.game_over_snd = pygame.mixer.Sound(
            "./assets/tractor_driving_game_over.wav"
        )
        # Set volume for sound effect in range 0.0 - 1.0
        pygame.mixer.Sound.set_volume(self.game_over_snd, .5)

        # Load and play background music
        pygame.mixer.music.load("./assets/tractor_driving.wav")

        # Set volume to 30%, range from 0.0 (mute) to 1.0 (full volume)
        pygame.mixer.music.set_volume(0.3)

        # Stop any other music from playing
        pygame.mixer.stop()

        # Play background game music in continious loop from the beginning
        pygame.mixer.music.play(-1)

        # Create font for scoring
        self.font_score = pygame.font.SysFont("Verdana", 20)
```

Game Over

This is a brand new method which uses the PyGame Menu library to create a Game Over menu.

```python
# ------------------------- DISPLAY GAME OVER ----------------------------
    def game_over(self):
        """Display game over on top of the stopped game"""
        # Stop background sound
        pygame.mixer.music.stop()

        # Play game_over music until the user clicks a button
        pygame.mixer.Sound.play(self.game_over_snd, loops=-1)

        # Define a menu object for the game over screen
        game_over = pm.Menu(
            title="Game over",      # Set title menu to "Game over"
            width=config.WIDTH,     # Set to width of game surface
            height=config.HEIGHT,   # Set to height of game surface
            # Set the theme of the menu to an orange color scheme
            theme=pm.themes.THEME_ORANGE
        )

        # Display final score
        game_over.add.label(f"Score: {self.score}")

        # Add label to provide space between buttons
        game_over.add.label("")

        # Add a button to the game over menu for exiting the game
        game_over.add.button(
            title="Play Again?",    # Button text
            action=main             # Call main() to start over
        )

        # Add label to provide space between buttons
        game_over.add.label("")

        # Add a button to the game over menu for exiting the game
        game_over.add.button(
            title="Exit",           # Button text
            action=pm.events.EXIT   # Exit the game when clicked
        )

        # Run the main loop of the game over menu on the specified surface
        game_over.mainloop(self.surface)
```

There are different themes you can choose for the game_over object. This example uses THEME_ORANGE. You can use any of the following to customize your menu.

```
THEME_BLUE
THEME_DARK
THEME_DEFAULT
THEME_GREEN
THEME_ORANGE
THEME_SOLARIZED
```

Update Ball

Modify the update_ball method. If the ball hits the bottom, game over.

```
# ------------------------- UPDATE BALL ----------------------------------#
    def update_ball(self):
        # Check for collision with left or right wall
        if self.ball_rect.left <= 0 or self.ball_rect.right >= config.WIDTH:
            # Reverse x direction multiply by -1
            self.speed_x = self.speed_x * -1

        # Check for collision with top or bottom wall
        if self.ball_rect.top <= 0 or self.ball_rect.bottom >= config.HEIGHT:
            # Reverse y direction multiply by -1
            self.speed_y = self.speed_y * -1

        # Move the ball position every frame
        self.ball_rect.x = self.ball_rect.x + self.speed_x
        self.ball_rect.y = self.ball_rect.y + self.speed_y

        # Ball hits bottom, player loses
        if self.ball_rect.bottom > config.HEIGHT:
            self.game_over()
```

Check Collision

The check collision method is changed.

```python
# ---------------------- CHECK COLLISION ----------------------------------#
    def check_collision(self):
        """Check for collision between two rects"""
        # The ball has to be above the tractor to collide
        # Does the ball collide with the tractor?
        # If so, reverse the ball y direction [1]
        if self.tractor_rect.colliderect(
            self.ball_rect
        ) and self.ball_rect.bottom < self.tractor_rect.top + 4:

            # Reverse y direction
            self.speed_y = self.speed_y * -1

            # Randomly change x direction
            direction = randint(0, 1)
            if direction == 0:
                self.speed_x = self.speed_x * -1

            # Increase speed by 10% each time the ball is hit
            self.speed_x = self.speed_x * 1.05
            self.speed_y = self.speed_y * 1.05

            # Increase score by 1
            self.score = self.score + 1
            pygame.mixer.Sound.play(self.ball_hit)
```

Draw the Score

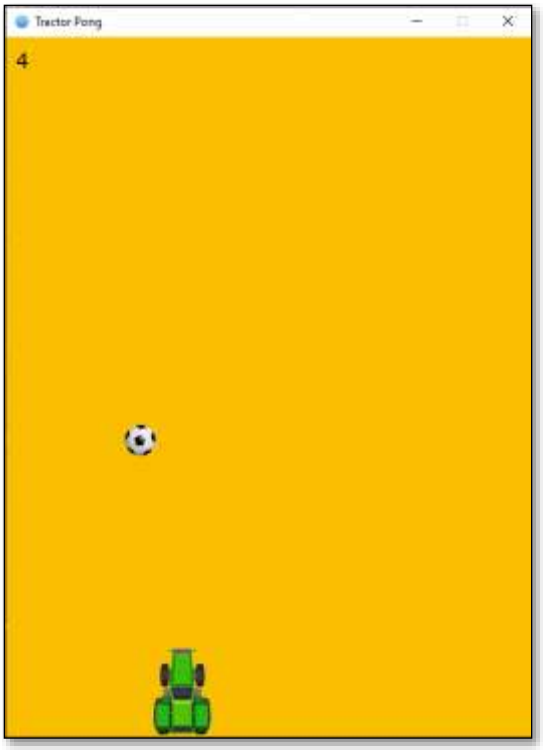
Draw the score on the screen.

```python
# --------------------------- DRAW ----------------------------------------#
    def draw(self):
        # Fill the display surface to clear the previous screen
        # Comment out this line to see why is is necessary
        self.surface.fill(config.COUGAR_GOLD)

        # Draw the ball on the backbuffer
        self.surface.blit(
            self.ball,          # Image to draw
            self.ball_rect      # Location to draw the image
        )

        # Draw the tractor on the backbuffer
        self.surface.blit(
            self.tractor,       # Image to draw
            self.tractor_rect   # Location to draw the image
        )

        # Render score before drawing it on the surface
        score_display = self.font_score.render(
            f"{self.score}",    # Score
            True,               # Antialiasing true
            "black"             # Font color
        )

        # Draw score on the surface
        self.surface.blit(score_display, (10, 10))

        # ------- COPY BACKBUFFER INTO VIDEO MEMORY -----------------------#
        # Copy the backbuffer into video memory
        pygame.display.update()

```

Add a main method to allow the game to start over.

```python
def main():
    # Initialize program object and start game
    tractor_pong = TractorPong()
    tractor_pong.game_loop()


main()
```

Example run:



The tractor is King.

What's Next?

- Change the colors to different RGB colors.
- Add more difficulty levels.
- Keep track of the highest score between games.
- Add more music, change the music
- Change out the images.
- Change the size of the playing field.
- Make the game your own.

Assignment Submission

1. Attach all tutorials and assignments.
2. Attach screenshots showing the successful operation of each tutorial program.
3. Submit in Blackboard.